

Part 2 — Hardware Implementation

Building the real O-RAN S-Plane timing testbed · Companion to the Project Proposal · Tanmay

Read first. The hardware below is **engineering and stronger validation of our method — not the novelty**. TIMESAFE already demonstrated the attack on real hardware. **Key insight:** because our problem is the S-plane (timing), the real testbed is mostly **networking / timing gear, NOT software-defined radios**. A real O-RU is only needed to physically show the base-station crash.

1. Tier A — Real PTP / SyncE timing lab (no radio; does the whole proposal for real)

Item	Purpose	Indicative cost*
2–3 PCs with PTP hardware-timestamping NICs (Intel I210/I225/E810)	O-DU (PTP slave + monitor), attacker, load generator	NIC \$30–\$300 each; reuse existing PCs
Timing-aware Ethernet switch (PTP boundary/transparent clock + SyncE, ITU-T G.8275.1)	Carries the fronthaul timing	budget w/ PTP ~\$500; telecom-grade \$1,500–\$8,000
PTP Grandmaster (GNSS-disciplined) OR a PC running ptp4l as GM + a GPSDO	Trusted reference clock	appliance \$2,000–\$10,000; DIY GPSDO+Pi \$150–\$400
GNSS receiver + antenna + PPS	GNSS reference; create GNSS-loss / holdover faults	\$50–\$300
SFP modules, precision cabling, network TAP / mirror port	Passive PTP capture	\$100–\$400
Time-error tester / reference (optional)	Measure real time error vs G.8273.2	rent/borrow; buy \$5k+

Tier A total: DIY/budget ≈ **\$800–\$3,000** reusing PCs; professional ≈ **\$6,000–\$18,000**.

2. Tier B — Optional: add real O-RAN radio (only to show the actual 2 s crash)

Item	Purpose	Indicative cost*
Real O-RU (7.2x split) + O-DU server	Full base-station demo	O-RU \$3,000–\$20,000+; server \$2,000–\$6,000
SDR + GPSDO (only if the radio air-interface is also wanted)	UE / RF side	USRP \$1,500–\$5,000

3. How it connects to our existing software

Our `oran_splane_selfhealing` code becomes the **monitor + decision** side, running on the O-DU PC. Instead of the Python simulator, it reads **real** PTP telemetry from **linuxptp** (`ptp4l` / `phc2sys` / `ts2phc`). The recovery actions map to **real commands**: switch the PTP source, enter holdover, or block a rogue master (via BMCA / an allow-list). A separate attacker PC runs the spoof / replay scripts. **Same software, real inputs.**

4. Software needed on the hardware

- **linuxptp** — `ptp4l` (PTP daemon), `phc2sys`, `ts2phc` (sync the PTP hardware clock).
- SyncE configuration; **tc netem** for controlled packet-delay-variation.
- **tcpdump** / **Wireshark** for passive capture; PTP-hardware-clock drivers (`/dev/ptpN`).

5. Setup requirements & considerations (things easy to overlook)

- PTP **profile** choice: ITU-T **G.8275.1** (full timing support) for the fronthaul.
- **Boundary-clock vs transparent-clock** switch — affects time-error budget.
- **MACsec** on the fronthaul link for transport security.
- GNSS **antenna sky view / roof access**; PPS **cable length & skew**.
- Holdover oscillator quality (**OCXO** > **TCXO**); lab **power + UPS**; **thermal stability** (timing drifts with temperature).

- A **golden reference clock** to measure true time error against.
- **RF licensing / shielding** only if the O-RU transmits (Tier B). **Tier A needs no radio and no spectrum licence.**